

Centro de Comercio y Servicios

Análisis y Desarrollo de Software

Instructor: Raúl Gómez

Estructuras repetitivas

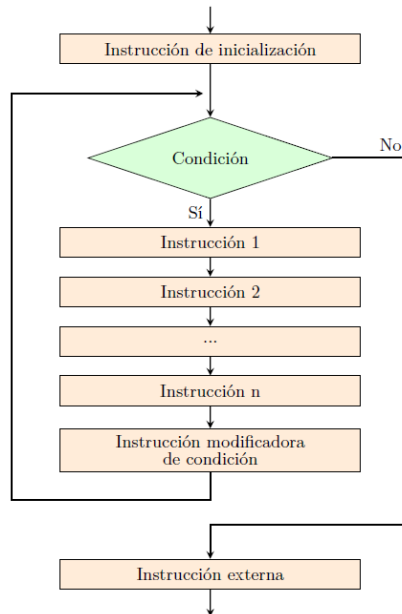
Es de utilidad para que determinados bloques de código se ejecuten varias veces mientras se cumpla una condición.

Existen 3 elementos que controlan el flujo del bucle. La inicialización que fija los valores con los que iniciamos el bucle. La condición de permanencia en el bucle y la actualización de las variables de control al ejecutarse la iteración.

Estructura mientras

La estructura repetitiva mientras (en inglés while o dowhile: hacer mientras) es aquella en que el cuerpo del bucle se repite mientras se cumple una determinada condición.

Diagrama de flujo



Pseudocódigo

```
vuelta = 0
Mientras vuelta < 100 Hacer
    Escribir vuelta
    vuelta = vuelta + 1
Fin Mientras
```

Sentencia while

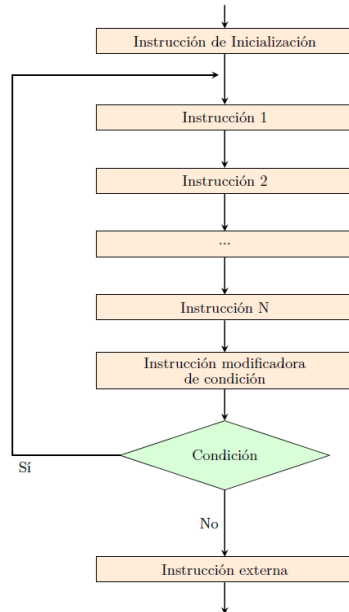
El bloque de código dentro del while se ejecutará mientras se cumpla la condición.

```
var vuelta = 0;  
  
Condición → while(vuelta < 100){  
    console.log(vuelta)  
    vuelta++ // vuelta = vuelta +1  
}
```

Estructura hacer-mientras

Es un ciclo condicionado al final, lo cual garantiza que sus instrucciones se ejecuten por lo menos una vez.

Diagrama de flujo



Pseudocódigo

```
dato = Falso
Repetir
    Escribir "Al menos una vez..."
Hasta Que Dato == Verdadero
```

Sentencia do/while

Primero ejecutará el código una vez y luego evaluará si debe repetirse.

```
let dato = false;  
  
do {  
  console.log("Al menos una vez...");  
} while(Dato)
```

Condición →

Sentencia do/while

Ejemplo:

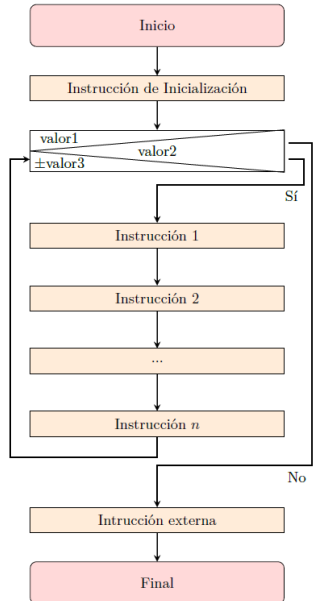
```
var i = 1;  
  
do {  
    console.log(i);  
    i++; // i = i+1  
} while (i < 11);
```

Condición →

Estructura desde/para

Es un ciclo condicionado al inicio, lo cual implica que su ejecución está determinada por la evaluación de su condición.

Diagrama de flujo



Pseudocódigo

```
Para v_numerica=v_inicial Hasta vuelta < 100 Con Paso vuelta = vuelta + 1  
Hacer  
    Escribir vuelta  
Fin Para
```


Sentencia for

Permite resumir en una línea la forma de un bucle while.

```
for(let vuelta = 0; vuelta < 100; vuelta++){  
  console.log(vuelta);  
}
```

Bucles anidados

Los bucles anidados constan de un bucle externo con uno o más bucles internos. Cada vez que se repite el bucle externo, los bucles internos se repiten, se reevalúan los componentes de control y se ejecutan todas las iteraciones requeridas.

Bucles anidados

Ejemplo:

El segmento de programa siguiente visualiza una tabla de multiplicación por cálculo y visualización de productos de la forma $x * y$ y para cada x en el rango de 1 a X_{ultimo} y desde cada y en el rango 1 a Y_{ultimo} (donde X_{ultimo} , y Y_{ultimo} son enteros prefijados). La tabla que se desea obtener es:

Bucles anidados

Ejemplo:

$$1 * 1 = 1$$

$$1 * 2 = 2$$

$$1 * 3 = 3$$

$$1 * 4 = 4$$

$$1 * 5 = 5$$

$$2 * 1 = 2$$

$$2 * 2 = 4$$

$$2 * 3 = 6$$

$$2 * 4 = 8$$

$$2 * 5 = 10$$

...

21/06/24

Bucles anidados

Ejemplo:

```
let producto;

for (let x = 1; x <= 5; x++) {
  for (let y = 1; y <= 10; y++) {
    producto = x*y;
    console.log(x + '*' + y + '=' + producto);
  }
}
```

Bucles anidados

Ejemplo:

```
for (let x = 1; x <= 5; x++) {  
  for (let y = 1; y <= 10; y++) {  
    producto = x*y;  
    console.log(x + '*' + y + '=' + producto);  
  }  
}
```

Bucle externo



Bucle interno



Bucles anidados

Ejemplo:

Escribir un programa que visualice un triángulo isósceles.

*

* * *

* * * * *

* * * * * * *

* * * * * * * *

Bucles anidados

```
let num_lineas = 5;
let espacio = '';
let asterisco = '';

for (let fila = 1; fila <= num_lineas; fila++) {
  for (let espacios = num_lineas-fila; espacios > 0; espacios--) {
    espacio += ' ';
  }

  for (let x = 1; x < 2*fila; x++) {
    asterisco += '*';
  }

  console.log(espacio + asterisco);

  espacio = '';
  asterisco = '';
}
```


Operadores de asignación

Los operadores de asignación actúan como una notación abreviada para expresiones utilizadas con frecuencia.

Operador	Sentencia abreviada	Sentencia no abreviada
<code>+=</code>	<code>m += n</code>	<code>m = m + n;</code>
<code>-=</code>	<code>m -= n</code>	<code>m = m - n;</code>
<code>*=</code>	<code>m *= n</code>	<code>m = m * n;</code>
<code>/=</code>	<code>m /= n</code>	<code>m = m / n;</code>
<code>%=</code>	<code>m %= n</code>	<code>m = m % n;</code>

Operadores de incremento y decremento

Los operadores ++ y —, denominados de incremento y decremento, suman o restan 1 a su argumento, respectivamente, cada vez que se aplican a una variable.

Incremento	Decremento
n++	n--
n += 1	n -= 1
n = n+1	n = n-1

Ejercicios prácticos

Ejercicio 1

Sumar los números enteros de 1 a 100 mediante:
a) sentencia do-while; b) sentencia while; c)
sentencia for.

Ejercicios prácticos 3

Ejercicio 2

Imprimir las treinta primeras potencias de 4, es decir, 4 elevado a 1, 4 elevado a 2, etc.

Ejercicios prácticos 3

Ejercicio 3

Imprimir todos los números primos entre 2 y 1.000 inclusive.

Ejercicios prácticos 3

Ejercicio 4

Escribir un programa que visualice el siguiente dibujo:

```

    *
  * * *
* * * * *
* * * * * *
* * * * * * * *
  * * * * * *
    * * * * *
      * * * *
        * * *
          *
```

Ejercicios prácticos 3

Ejercicio 5

Imprimir los cuadrados de los enteros de 1 a 20.

Ejercicios prácticos 3

Ejercicio 6

Visualizar en pantalla una figura similar a la siguiente:

*

**

Ejercicios prácticos 3

Ejercicio 7

Escribir un programa que visualice la siguiente salida:

1

1 2

1 2 3

1 2 3 4

1 2 3

1 2

1

21/06/24

Ejercicios prácticos 3

Ejercicio 8

Diseñar e implementar un programa que solicite a su usuario un valor no negativo n y visualice la siguiente salida ($n = 6$):

1 2 3 4 5 6

1 2 3 4 5

1 2 3 4

1 2 3

1 2

1

21/06/24

Ejercicios prácticos 3

Ejercicio 9

Diseñar un algoritmo que sume los $m = 30$ primeros números pares.

Ejercicios prácticos 3

Ejercicio 10

Diseñar un programa que produzca la siguiente salida:

ZYXWVTSRQPONMLKJIHGFEDCBA

YXWVTSRQPONMLKJIHGFEDCBA

XWVTSRQPONMLKJIHGFEDCBA

WVTSRQPONMLKJIHGFEDCBA

VTSRQPONMLKJIHGFEDCBA

...

FEDCBA

EDCBA

DCBA

CBA

BA

A

Ejercicios prácticos 3

Ejercicio 11

Genere e imprima los múltiplos de 3 que se encuentren entre 6 y n , donde n tiene que ser superior a 6.

Ejercicios prácticos 3

Ejercicio 12

Elabore un algoritmo, que genere e imprima las letras del abecedario de la siguiente forma:

```
Z
Z Y
Z Y X
Z Y X W
Z Y X W V
Z Y X W V U
Z Y X W V U T
Z Y X W V U T S
Z Y X W V U T S R
Z Y X W V U T S R Q
Z Y X W V U T S R Q P
Z Y X W V U T S R Q P O
Z Y X W V U T S R Q P O N
Z Y X W V U T S R Q P O N M
Z Y X W V U T S R Q P O N M L
Z Y X W V U T S R Q P O N M L K
Z Y X W V U T S R Q P O N M L K J
Z Y X W V U T S R Q P O N M L K J I
Z Y X W V U T S R Q P O N M L K J I H
Z Y X W V U T S R Q P O N M L K J I H G
Z Y X W V U T S R Q P O N M L K J I H G F
Z Y X W V U T S R Q P O N M L K J I H G F E
Z Y X W V U T S R Q P O N M L K J I H G F E D
Z Y X W V U T S R Q P O N M L K J I H G F E D C
Z Y X W V U T S R Q P O N M L K J I H G F E D C B
Z Y X W V U T S R Q P O N M L K J I H G F E D C B A
```

Ejercicios prácticos 3

Ejercicio 13

El factorial de un número, es el producto obtenido al multiplicar un número dado por todos los enteros positivos sucesivos inferiores. Además, se tiene establecido que los números negativos no poseen factorial y que el factorial de 0 es 1. El factorial se representa con un signo de exclamación precedido de un número (n!).

Por ejemplo:

$$7! = 7*6*5*4*3*2*1 = 5040$$

o se puede expresar como:

$$7! = 1*2*3*4*5*6*7 = 5040.$$

A 7! se le denomina 7 factorial, también es llamado el factorial de 7.

Teniendo en cuenta el anterior contexto, diseñe un programa que calcule el factorial de un número entero.

Ejercicios prácticos 3

Ejercicio 14

Ejecutar y visualizar el programa siguiente que imprime una tabla de m filas por n columnas y un carácter prefijado. (El usuario proporciona estos datos desde consola)

Ejercicios prácticos 3

Ejercicio 15

Escriba un programa JavaScript para construir el siguiente patrón de caracteres utilizando un ciclo for.

```
*  
  
**  
  
***  
  
****  
  
*****  
  
****  
  
***  
  
**  
  
*
```